GreenVerifier: Final Report

University of Southern California

Alexis Moumtzis
moumtzis@usc.edu

Ray Zhang
zhangray@usc.edu

Abstract

Environmental, Social and Governance (ESG) reports contain large volumes of unstructured narrative disclosures that are difficult to verify or audit systematically. GreenVerifier proposes an automated verification framework that combines retrieval-augmented transformers with supervised classification to determine whether a given ESG claim is Supported or Contradicted, while identifying the underlying textual or numerical evidence. We compile a multi-industry dataset of corporate sustainability and TCFD reports, develop a preprocessing and retrieval pipeline using and train a DeBERTa cross-encoder for fine-grained claim–evidence validation. We evaluate our approach against several large language model baselines used in zero-shot and few-shot settings. Our preliminary findings show that while the supervised retrieval-augmented model performs competitively, generic LLMs remain strong baselines, highlighting both the difficulty of the verification task and the need for further domain-specific optimization.

The full implementation of GREENVERIFIER, including preprocessing, retrieval, training, and evaluation scripts, is available at: https://github.com/zhang-yuhui/green-verifier.git

1 Introduction

GREENVERIFIER introduces a retrieval-augmented framework for automated verification of ESG statements. Given a claim such as "Scope 1+2 emissions fell 42% vs 2019," and the corresponding company report, the system classifies it as *Supported* or *Contradicted* while retrieving the relevant textual or numerical evidence. By grounding verification in report content, GREENVERIFIER aims to transform narrative ESG disclosures into consistent, auditable outputs.

This work is guided by three hypotheses: (1) retrieval improves the grounding and consistency of verification decisions; (2) supervised cross-encoder models can better capture ESG-specific terminology and numerical patterns; and (3) large general-purpose LLMs remain competitive baselines, highlighting the difficulty of surpassing them without domain-specific adaptation.

2 Related Work

Early work on automated claim verification has been formalized through benchmark datasets such as FEVER (Thorne et al., 2018), which frame the task as determining whether a claim is supported, refuted, or unverifiable based on evidence retrieved from a large corpus.

Early work on claim verification such as the UKP-Athene system (Hanselowski et al., 2018) explicitly addressed multi-sentence textual entailment by aggregating evidence across multiple retrieved passages. Their approach demonstrated that reliable fact verification often requires synthesizing complementary evidence fragments rather than relying on a single supporting sentence.

Schimanski et al. (2024) used text-mining and topic modeling to quantify linguistic patterns in ESG communication. Their work highlighted reporting heterogeneity, motivating GREENVERIFIER's standardized text preprocessing and chunking for consistent semantic representation.

Zou et al. (2025) introduced ESGREVEAL, a retrieval-augmented system for extracting structured KPIs from sustainability reports. Its architecture inspired the RAG-based retrieval backbone in GREENVERIFIER, which grounds claims in contextually relevant report sections. Li et al. (2025) further demonstrated that domain-specific LLM fine-tuning improves classification in ESG texts, supporting our choice to train on ESG-tailored embeddings for improved accuracy.

Verification-focused work such as Seki et al. (2024) proposed the ML-Promise dataset for

corporate promise verification, showing that retrieval-augmented generation enhances factual consistency. This influenced GREENVERIFIER's claim–evidence pairing and validation design. Finally, Zhang et al. (2025) surveyed greenwashing detection methods, reinforcing the need for transparent verification frameworks.

GREENVERIFIER extends these efforts by integrating RAG-based retrieval and supervised classification to verify ESG claims as *Supported* or *Contradicted*, linking each prediction to its textual and numerical evidence.

3 Methodology

As seen in Figure 1, the system follows a two-phase architecture designed for long-document claim verification, inspired by Retrieval-Augmented Generation (RAG) paradigms (Patrick Lewis, 2020). First, each report is segmented into overlapping text chunks that preserve local context. A bi-encoder model independently encodes claims and document chunks into a shared vector space, enabling efficient similarity search. Given a claim, we compute its embedding and perform a k-nearest-neighbor search over the chunk index to identify the most relevant candidate evidence passages. This retrieval step dramatically reduces the search space and ensures that downstream reasoning focuses only on contextually meaningful segments.

Thereafter, each retrieved chunk is paired with the claim and passed to a cross-encoder model. Unlike the bi-encoder, which embeds claim and chunk independently, the cross-encoder jointly processes the pair and produces label scores indicating whether the chunk supports, contradicts, or fails to provide evidence for the claim. The final prediction is obtained by aggregating scores across all candidate chunks and selecting the label with the highest confidence, along with the corresponding evidence passage. This combines the scalability of embedding-based retrieval with the fine-grained reasoning ability of attention-based classification, enabling efficient and interpretable verification over long ESG reports.

4 Dataset

To build and evaluate GREENVERIFIER, we assembled a corpus of 126 corporate sustainability and TCFD reports spanning multiple sectors and reporting formats. These documents were sourced from official company sustainability webpages and the SEC EDGAR archive (U.S. Securities and Exchange Commission, 2025). to ensure diversity in structure, terminology, and disclosure quality. Each report was uploaded to a cloud storage bucket and its metadata—company name, year, sector, source URL and document type was automatically indexed in a cloud-based database. This setup enables reproducibility and direct linkage between report content and its derived annotations.

4.1 Claim Generation

To create training examples, we generated claim–evidence pairs using a Llama 3 model accessed through the Hugging Face Inference API. Each report, truncated to a fixed maximum length, was provided to the model with an instruction to propose factual claims, assign a label (*support* or *not support*), and extract a short supporting or contradicting evidence snippet. Generation is performed without retrieval, allowing the LLM to process the document holistically and produce self-contained claims grounded in the visible report content. Each document yields approximately five to ten claims, which are stored in structured JSON files together with source metadata for reproducibility.

This process follows the paradigm of *synthetic supervision*, where large language models are used to generate labeled training data directly from raw text, as formalized in the SELF-INSTRUCT framework (Wang et al., 2022b). By leveraging Llama 3 to bootstrap ESG-specific claims and labels, we obtain scalable weak supervision while maintaining explicit evidence grounding.

4.2 Quality Control and Splitting

To prevent leakage and ensure a fair evaluation, dataset splitting was performed at the company level: all reports and associated claims from a given company belong exclusively to either the training (70%), validation (15%), or test (15%) set. This avoids the model benefiting from repeated phrasing or consistent numeric patterns across a single firm's reports.

The validation set underwent manual review. Each generated claim was checked for factual alignment, numerical correctness, and contextual appropriateness. To stress-test the system, we additionally introduced "edge-case" claims involving ambiguous phrasing, near-duplicates and metrics requiring contextual interpretation.

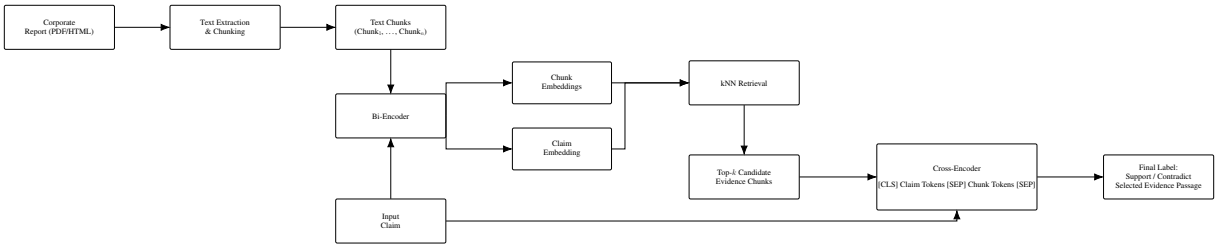

Figure 1: Overview of the retrieval and verification pipeline. The report is converted to text chunks that, together with the claim, are encoded by a shared bi-encoder into chunk and claim embeddings. A k-nearest-neighbor search over these embeddings retrieves top-k candidate evidence chunks, which are combined with the claim in a cross-encoder to predict a verification label and select supporting evidence.

5 Model Architecture

The proposed system for ESG claim verification adopts a multi-stage architecture inspired by Retrieval-Augmented Generation (RAG) principles. To handle both unstructured narrative text and structured XBRL Key Performance Indicators (KPIs), the model integrates encoder-only transformer modules for retrieval and verification, extended with a decoder-based component for rationale generation.

5.1 Stage 0: Pre-processing and Indexing

Each report is first converted from PDF or HTML into raw text and then segmented into overlapping passages of 256–512 tokens to ensure semantic continuity and avoid boundary effects during retrieval. In addition to unstructured narrative text, structured disclosures such as XBRL and iXBRL facts are parsed and transformed into canonical natural-language pseudo-sentences (e.g., *"Research and development expense, 2022: USD 6.26 billion"*), enabling the system to treat tabular and numeric information in the same retrieval space as ordinary text.

This pre-processing stage yields two parallel sources of evidence. The first is a sequence of semantic text chunks extracted from the report body, while the second is a set of structured KPI statements derived from tables or tagged numeric values. These complementary sources ensure that both narrative explanations and quantitative values can be retrieved at verification time.

Accordingly, two independent vector indices are created:

- **Text Index**: containing embedded text chunks from the main body of the report.
- **KPI Index**: containing embedded numeric or structured facts derived from XBRL/iXBRL and tabular disclosures.

A practical challenge in sustainability reports is that the same KPI or concept is expressed through heterogeneous surface forms (e.g., "Scope 1+2", "Scopes 1 and 2", "direct and energy indirect emissions", or metric names that vary across companies). To reduce retrieval brittleness, we add a lightweight normalization layer during preprocessing that standardizes common ESG entities and units. Concretely, we canonicalize numeric patterns (percentages, currency amounts, and year references), normalize unit expressions (e.g., tCO_2e variants), and map frequent ESG abbreviations (e.g., "S1/S2/S3", "GHG", "tCO2e") into a consistent textual representation. This transformation is applied both to indexed report chunks and to claims before embedding, improving lexical and semantic alignment without requiring additional supervision.

All entries are embedded using a shared bi-encoder model, and the resulting vectors—together with metadata such as the source file, chunk ID, and fact type—are stored in a Qdrant similarity-search database to support fast retrieval during verification. A detailed description of the full pre-processing pipeline, including OCR fallback, table extraction and XBRL parsing logic, is provided in the Appendix A.

5.2 Stage 1: Evidence Retrieval

The retrieval stage relies on dense text embeddings trained to align queries and passages in a shared semantic space. We adopt an E5-style bi-encoder embedding model, whose training objective is based on weakly supervised contrastive learning over large-scale text pairs (Wang et al., 2022a). This approach has been shown to produce highly effective representations for semantic retrieval without requiring task-specific supervision.

To enable scalable retrieval over long documents, we employ a bi-encoder architecture in which claims and report passages are encoded independently into a shared embedding space, following the Sentence-BERT framework (Reimers and Gurevych, 2019). Given a claim, the retrieval component locates the most relevant evidence passages from both indices. A dual-encoder transformer (`e5-base`) encodes the claim and all indexed items independently. Cosine similarity is used to perform a k-nearest neighbor search, returning the top-k candidates. In our experiments, we set $k = 5$, as larger values provided only marginal accuracy improvements while introducing significant computational overhead during training.

$$E_k = \{e_1, e_2, \ldots, e_k\}, \quad e_i \in Report or KPI index.$$

This stage corresponds to the retrieval component of Retrieval-Augmented Generation (RAG), ensuring that only the most semantically relevant report excerpts are passed to the downstream verification model.

5.3 Stage 2: Verification and Classification

Each retrieved candidate (e_i) is paired with the claim (c) and processed by a cross-encoder transformer (`deberta-v3-base`). The model computes class probabilities for each pair:

$$P(y \mid c, e_i) = softmax(Wh_{[CLS]} + b),$$

where $y \in \{support, notsupport\}$.

The final label is determined by aggregating scores across all evidence candidates:

$$\hat{y} = \arg\max_{y} \max_{i} P(y \mid c, e_i).$$

The evidence with the highest confidence is stored as the attribution reference for interpretability.

For the verification model, we adopt DeBERTa-v3 rather than the classical BERT architecture due to its significantly stronger performance on sentence-pair reasoning tasks, which are central to claim–evidence verification. Unlike BERT, DeBERTa introduces disentangled attention, separating content and positional embeddings so the model can learn relational structures more effectively—an essential property when comparing a claim to a semantically complex report passage. It further incorporates enhanced mask decoders and larger pretraining corpora, enabling better generalization in domain-specific settings such as ESG text. Empirical results reported by Pengcheng He (2021) show that DeBERTa variants substantially outperform BERT on GLUE, SuperGLUE, and natural language inference benchmarks, making it a more suitable backbone for fine-grained pairwise classification. These architectural advantages translate directly into higher accuracy and more stable optimization when evaluating support or contradiction between a claim and retrieved evidence.

During training, we treat all candidate chunks associated with a given claim (retrieved top-k passages plus a small number of randomly sampled negatives from the same report) as a *bag*. The cross-encoder is applied independently to each (claim, chunk) pair, producing instance-level logits $z_{ii \in B}$ for a claim-level label y^*. Instead of supervising each instance directly, we adopt a *Multiple Instance Learning* (MIL) framework (Maximilian Ilse, 2018), which models weak supervision over sets of instances. Specifically, we apply a differentiable soft MIL pooling operator that aggregates instance scores into a single bag-level representation:

$$\tilde{z} = \log \sum_{i \in B} \exp(z_i),$$

and define the loss as the cross-entropy between $\tilde{z}$ and the bag label y^*:

$$\mathcal{L} = CE(\tilde{z}, y^*).$$

This *soft pooling* encourages at least one chunk in a positive bag to receive a high score, while remaining numerically stable and allowing gradients to flow through all candidates.

To improve stability and reduce compute, we adopt a partial fine-tuning strategy: most encoder layers of the pretrained cross-encoder are frozen, while only the classification head and the top few transformer layers are updated during training. Gradients are accumulated over multiple bags to simulate a larger effective batch size, and validation performance is monitored after each epoch to detect overfitting.

In practice, several hyperparameters play a critical role in balancing verification performance, training stability, and computational cost. The number of retrieved candidates per claim is fixed to $k = 5$, which we found to provide a favorable trade-off between recall of relevant evidence and the introduction of noisy or weakly related chunks. Larger values of k slightly increase the probability that

supporting evidence is present in the bag, but also expand bag size, dilute gradients under soft MIL pooling and significantly increase training time. Conversely, smaller values make the system more sensitive to retrieval errors, placing a hard upper bound on end-to-end verification accuracy.

Negative sampling is controlled by adding $n = 2$ randomly selected chunks from the same report to each bag. Sampling negatives from the source document preserves topical similarity and yields substantially harder negatives than corpus-level sampling. This encourages the model to learn fine-grained semantic distinctions rather than rely on superficial lexical overlap. Increasing the number of negatives beyond this point resulted in diminishing returns, as larger bags tend to flatten the pooled score distribution and slow convergence.

Optimization hyperparameters are tuned to accommodate the expanded instance-level representation induced by the MIL framework. Training is performed with a small number of bags per step and gradient accumulation to simulate a larger effective batch size without exceeding GPU memory constraints. A low learning rate of 1×10^{-5} combined with AdamW optimization and weight decay (0.01) was necessary to maintain stable updates when only a subset of model parameters is trainable. Validation performance is monitored after each epoch to detect overfitting, which is particularly important given the limited dataset size and weakly supervised training signal.

5.4 Evidence Attribution and Alignment with RAG Principles

The system ensures transparent and interpretable predictions by logging the specific evidence segment that drives each classification decision. Because the retriever ranks candidate evidence, the verification model can automatically associate the final predicted label with the highest-scoring report chunk or KPI fact, providing full traceability between outputs and source material. Although the pipeline uses encoder-only architectures rather than a generative model, it remains consistent with Retrieval-Augmented Generation (RAG) principles (Patrick Lewis, 2020): retrieval functions as a domain-specific knowledge base, while the cross-encoder verification model acts as the downstream reasoning component. Together, these mechanisms create an interpretable, retrieval-grounded decision process tightly aligned with RAG methodology.

6 Baseline Model

6.1 Setup

For comparative analysis, we employ outputs from general-purpose large language models (LLMs) as baselines. Specifically, we utilize two distinct models: a smaller-scale variant, `Qwen/Qwen3-4B-Instruct-2507`, and an API-based model, `GPT-5.1-nano`. Both baseline configurations incorporate Retrieval-Augmented Generation (RAG) with `intfloat/e5-base-v2` serving as the embedding model.

To ensure a fair comparison, retrieval is constrained to the same document-level evidence pool used by GREENVERIFIER, preventing access to external knowledge or cross-document information. This setup isolates the effect of the reasoning and aggregation capabilities of LLMs, rather than differences in retrieval coverage.

6.2 Experimental Procedure

We apply Few-Shot Chain-of-Thought (CoT) prompting for both models. For each query, the model receives three exemplars accompanied by supporting evidence, after which it is prompted to generate intermediate reasoning steps prior to producing a final classification. The generated outputs are then parsed and compared against ground-truth labels to compute performance metrics.

In cases where models produce invalid outputs (e.g., generation failures, refusal responses, or format inconsistencies), predictions are treated as incorrect and assigned the opposite value of the true label. This conservative evaluation avoids inflating baseline performance and ensures that reported results reflect reliable verification behavior.

7 Performance and Results

The performance of the three models was systematically evaluated across four standard metrics: accuracy, precision, recall, and F1-score. The results are presented in Table 1.

It should be noted that the evaluation protocols differ across model families. The encoder model is evaluated at the *bag level*, where a prediction is considered correct if the aggregated bag label aligns with the ground truth. Conversely, LLM-based models are evaluated at the *claim level*, where correctness is determined by the exact match between the generated label (Supported or Contradicted) and the annotated reference label.

Model	Accuracy	Precision	Recall	F1-Score
DeBERTa Cross-Encoder	0.8094	0.8670	0.8030	0.8338
Qwen3-4B-Instruct-2507	0.8416	0.9408	0.7833	0.8548
GPT-5-nano	0.9000	0.9810	0.8512	0.9115

Table 1: Comprehensive performance comparison of all models across evaluation metrics.

7.1 Accuracy

GPT-5-nano demonstrates the highest accuracy at 0.9000, substantially outperforming both Qwen3-4B-Instruct-2507 (0.8416) and the DeBERTa baseline (0.7930). This indicates that GPT-5-nano correctly predicts claim labels across the dataset with greater frequency. The accuracy metric, computed as the ratio of correct predictions to total evaluated claims, reflects overall classification performance and provides an intuitive measure of model effectiveness.

7.2 Precision

Precision measures the reliability of positive predictions, defined as:

$$Precision = \frac{TP}{TP + FP}$$

where TP denotes true positives (correct "support" predictions) and FP denotes false positives (incorrect "support" predictions). GPT-5-nano achieves the highest precision (0.9810), indicating that when it predicts a claim as supported, this prediction is correct in approximately 98% of cases. Qwen3-4B-Instruct-2507 also demonstrates strong precision (0.9408), while the DeBERTa encoder exhibits comparatively lower precision (0.7511), suggesting a higher false positive rate.

7.3 Recall

Recall quantifies the model's ability to identify truly supported claims:

$$Recall = \frac{TP}{TP + FN}$$

where FN denotes false negatives (missed "support" cases). The DeBERTa encoder achieves the highest recall (0.8320), successfully identifying approximately 83% of genuinely supported claims. GPT-5-nano attains a recall of 0.8512, while Qwen3-4B-Instruct-2507 exhibits the lowest recall at 0.7833. This metric is particularly important for applications where failing to identify supported claims carries significant consequences.

7.4 F1-Score

The F1-score provides a harmonic mean of precision and recall, offering a balanced evaluation metric especially valuable in scenarios with class imbalance:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

GPT-5-nano achieves the highest F1-score (0.9115), indicating superior overall performance by balancing both precision and recall. Qwen3-4B-Instruct-2507 follows with an F1-score of 0.8548, while the DeBERTa baseline achieves 0.8190. The F1-score results suggest that GPT-5-nano provides the most reliable balance between avoiding false positives and false negatives, making it the most suitable model for practical deployment in claim verification tasks.

8 Analysis of the Results

8.1 Summary of Findings

Our empirical results indicate that general-purpose large language models used as zero-shot or lightly prompted baselines outperform the current implementation of GREENVERIFIER on overall verification accuracy. This outcome highlights the impact of large-scale pretraining in modern LLMs, which enables them to implicitly aggregate dispersed information and infer relationships across distant sections of a document. In contrast, GREENVERIFIER explicitly constrains reasoning to retrieved evidence segments, prioritizing transparency and attribution. While this design choice improves interpretability, it also introduces additional sources of error that affect end-to-end performance.

8.2 Hypothesis Evaluation

We conclude by revisiting the hypotheses stated in the Introduction (Section 1). First, our results partially support the claim that retrieval improves grounding and consistency. While retrieval does not yield the highest overall label accuracy compared to large LLM baselines, it consistently enables evidence-linked decisions, and correct pre-

dictions reliably map to the correct report region, which directly supports the grounding objective.

Second, the hypothesis that supervised cross-encoder training improves ESG-specific verification is supported in the sense that DeBERTa learns to discriminate highly similar ESG passages under hard negative sampling and weak supervision. However, the performance gap to LLM baselines suggests that model capacity and multi-chunk reasoning remain limiting factors that training alone does not fully overcome.

Third, the hypothesis that general-purpose LLMs remain strong baselines is strongly confirmed. Even with lightweight prompting, LLMs outperform the supervised pipeline on overall metrics, indicating that large-scale pretraining provides substantial advantages for implicit aggregation and long-context reasoning in ESG verification.

8.3 Evidence Attribution Quality

Following prior work on evidence-grounded verification (Thorne et al., 2018), we explicitly evaluate whether the selected evidence overlaps with gold reference annotations., which is a central design objective of GREENVERIFIER. For each claim, the model selects a single evidence passage corresponding to the highest-confidence chunk after MIL aggregation. We measure an *evidence hit rate*, defined as whether the selected chunk overlaps with the gold reference annotation used during dataset construction.

A prediction is counted as a hit if the model-selected chunk matches the gold chunk identifier or falls within a local neighborhood of ± 1 adjacent chunk, accounting for boundary effects introduced by fixed-size chunking. This metric evaluates whether the model grounds its decision in the correct region of the source document, independent of the final classification label.

Importantly, we observe that whenever the model predicts the claim label correctly, the associated evidence snippet consistently corresponds to the correct supporting or contradicting passage in the source document. This indicates that classification errors are primarily driven by retrieval or aggregation failures, rather than incorrect evidence attribution once the decision itself is correct.

8.4 Error Decomposition and Failure Modes

To better understand the performance gap between GREENVERIFIER and large LLM baselines, we conduct a qualitative decomposition of verification errors. We categorize incorrect predictions into four dominant failure modes, based on manual inspection of misclassified examples.

Retrieval Misses. In a non-trivial fraction of errors, the correct supporting or contradicting evidence does not appear in the top-k retrieved candidates. In these cases, verification failure is inevitable regardless of cross-encoder quality, highlighting the retrieval component as a hard bottleneck for end-to-end accuracy.

Chunk Boundary Effects. Some claims require evidence that spans multiple adjacent or non-adjacent chunks (e.g., a definition in one section and a numerical value in another). Because the verification model evaluates each chunk independently, such fragmented evidence cannot be jointly reasoned over, leading to false negatives in otherwise correct claims.

Numerical and Methodological Ambiguity. Certain claims hinge on subtle distinctions between absolute values, intensity metrics, restatements, or boundary definitions (e.g., market-based vs. location-based emissions). Even when relevant chunks are retrieved, the cross-encoder may fail to resolve these distinctions without explicit numerical reasoning support.

Linguistic Edge Cases. Finally, claims involving hedging language, forward-looking statements, or implicit assumptions (e.g., "on track," "aligned with," or "consistent with targets") pose challenges for binary verification. These cases often lack a single definitive evidence span, limiting the effectiveness of chunk-level supervision.

This analysis suggests that many observed errors arise from structural limitations of chunk-based retrieval and independent verification, rather than from incorrect evidence attribution when the relevant information is present.

9 Conclusions and Future Work

9.1 Limitations of Chunk-Based Verification

A central limitation of the proposed pipeline lies in its reliance on fixed-size text chunking. Although chunking is necessary to enable scalable processing of long ESG reports, it restricts the model's ability to jointly reason over information that spans multiple, non-adjacent segments. In practice, many ESG disclosures—particularly those involving emissions trajectories, long-term targets, conditional commitments, or methodology explanations—are distributed across several sec-

tions of a report. Because the verification stage evaluates each chunk independently and aggregates scores only at the decision level, the model cannot explicitly model interactions between complementary pieces of evidence.

This limitation becomes especially apparent in *edge cases*, where no single chunk contains sufficient information to fully support or contradict a claim. In such cases, correct verification would require synthesizing partial evidence from multiple passages, a capability that is largely absent from the current architecture. As a result, claims that are conceptually correct but whose evidence is fragmented across the document are disproportionately challenging for the system and contribute meaningfully to observed errors during evaluation.

9.2 Model Capacity and Retrieval Sensitivity

Another constraint arises from the size and capacity of the cross-encoder used for verification. While DeBERTa-v3 provides strong performance for sentence-pair classification, it remains significantly smaller than the LLM baselines evaluated in this work. This limits its ability to perform complex semantic reasoning, numerical comparison, and implicit aggregation of context. Although partial fine-tuning and Multiple Instance Learning help mitigate data sparsity, the verification model remains highly sensitive to upstream retrieval quality. If the correct evidence does not appear among the top-k retrieved candidates, verification inevitably fails, placing a hard upper bound on achievable performance.

Additional limitations include the relatively modest size of the labeled dataset, which constrains the diversity of claim formulations encountered during training, as well as the binary label space, which does not differentiate between genuinely contradictory evidence and cases of insufficient or ambiguous disclosure.

9.3 Future Directions

Despite these limitations, GREENVERIFIER offers several advantages that motivate continued development. Unlike black-box LLM baselines, the system provides explicit evidence attribution, enabling auditability and alignment with regulatory and compliance requirements. Moreover, its modular design allows individual components to be improved or replaced without retraining the entire pipeline.

Prior work on multi-sentence entailment for claim verification (Hanselowski et al., 2018) suggests that explicit modeling of cross-evidence interactions is a promising direction for overcoming the limitations of chunk-level verification. Future work could explore larger and more expressive cross-encoders, as well as hierarchical or graph-based evidence aggregation mechanisms that explicitly support multi-chunk reasoning. Incorporating structured numerical reasoning modules and expanding the dataset with manually verified, multi-evidence claims would further improve robustness, particularly in edge-case scenarios. Finally, scaling the pipeline to additional reporting standards, languages, and continuous reporting workflows represents a promising direction for practical ESG verification systems.

Overall, while LLM baselines currently achieve stronger raw performance, this work demonstrates the feasibility and importance of retrieval-grounded verification for long-document ESG analysis and establishes a foundation for scalable, transparent, and auditable claim verification.

A Appendix

In this section we will describe in more details the models and training methods used in this work.

A.1 Preprocessing and Index Construction

This appendix details the preprocessing and indexing pipeline used to convert raw ESG reports (PDF/HTML) into two searchable vector indices: (i) a narrative **Text Index** containing chunked report passages, and (ii) a **KPI Index** containing structured, numerically grounded pseudo-sentences derived from tables, numeric lines, and iXBRL facts. The goal is to support evidence retrieval over long, heterogeneous documents while preserving traceability through stable identifiers and rich metadata payloads.

Format-Specific Text Extraction. The pipeline supports two primary document types: PDF and HTML.

PDF Reports. For PDF inputs, we use `pdfplumber` to extract page-level selectable text. The extracted text is concatenated into a single report string. In parallel, we attempt to extract structured tables from each page using `page.extract_tables()`, converting them into row/column/value statements. This dual extraction strategy targets both narrative paragraphs and quantitative disclosures that are often presented as tables.

HTML Reports. For HTML inputs, we apply a structured parsing strategy using BeautifulSoup. The report text is extracted using `soup.get_text(separator="\n")`. Tables are parsed explicitly by iterating over `<table>` and `<tr>` elements and constructing row/column/value statements. If structured parsing fails (e.g., malformed HTML), a regex-based fallback strips tags and collapses whitespace to yield plain text.

OCR Fallback for Image-Heavy PDFs. Some ESG reports are scanned or image-heavy and contain little selectable text. To address this, the pipeline includes an optional OCR fallback using AWS Textract (`AnalyzeDocument`) with table detection enabled.

OCR is triggered only when two conditions are met: (i) the `ENABLE_TEXTRACT_OCR` flag is enabled, and (ii) the document size is within the synchronous Textract limit (5 MB). To avoid unnecessary OCR, an empirical text-density heuristic is applied by computing the average number of characters per page. Documents falling below a predefined threshold (`TEXTRACT_MIN_CHARS_PER_PAGE`) are treated as image-heavy.

Textract outputs are integrated in two ways: extracted `LINE` blocks are appended to the base text, and extracted `CELL` blocks containing digits are converted into KPI-style pseudo-sentences and added to the KPI Index.

Table-to-Sentence Transformation. To enable uniform retrieval across narrative and tabular content, extracted tables are converted into canonical pseudo-sentences. For PDFs, the first row is treated as the header and subsequent rows as records. Each numeric cell is mapped to a compact representation of the form:

```
Table p{page}, {row_label}, {col_label}:
{cell_value}
```

Row and column labels are inferred from the table structure when available, or synthetically generated otherwise. The same transformation is applied to HTML tables and OCR-derived tables. This representation preserves weak structural context while remaining compatible with dense embedding models.

KPI Fact Construction. The KPI Index is built from three complementary sources of numerically salient evidence.

iXBRL Facts (EDGAR HTML). For EDGAR-originating HTML with embedded iXBRL, we parse structured facts from `ix:nonfraction` and `ix:nonnumeric` tags. To enrich facts with temporal and unit context, the parser builds two maps: (i) a `context_map` extracted from `xbrli:context` elements, including period start/end (or instant) and optional dimensional members; and (ii) a `unit_map` extracted from `xbrli:unit` elements, including currency or measurement units (e.g., `iso4217:USD`). Each fact is converted into a canonical sentence, conditionally including period and unit information, e.g.,

```
Concept for the period from 2022-01-01
to 2022-12-31 has value 6.26 USD.
```

This yields machine-searchable KPI statements while retaining the accounting context required for verification.

Table-Derived Numeric Statements. All row/column/value pseudo-sentences produced from PDF/HTML/Textract tables are added as KPI facts with `fact_kind=table`. These facts are particularly important for ESG metrics that are predominantly disclosed in tables (emissions inventories, energy, water, workforce breakdowns, etc.).

Numeric-Line Fallback from Narrative Text. Finally, to capture numerically relevant evidence that is not part of a clean table or iXBRL structure, we apply a lightweight regex heuristic over the full extracted text to select lines likely to contain KPI-like values (years, percentages, or thousands separators). Each matching line is normalized and wrapped into a sentence template:

```
Reported value: {line},
```

with `fact_kind=numeric`. This fallback is deliberately conservative (capped to a fixed number of lines) to avoid overwhelming the KPI Index with noise while still improving recall for numeric evidence.

To ensure bounded compute and predictable index size, KPI sentences are capped per report (`MAX_KPI_SENTENCES_PER_REPORT`).

Tokenization and Chunking for the Text Index. Narrative report text is tokenized using a whitespace tokenizer and segmented using a sliding window of 400 tokens with an overlap of 100 tokens.

This overlap mitigates boundary effects where relevant evidence spans adjacent chunks.

$$\{\texttt{Chunk}_1, \ldots, \texttt{Chunk}_n\},$$

which form the searchable units in the Text Index.

Embedding Model and Indexing. Both the Text and KPI indices use a shared bi-encoder, `intfloat/e5-base-v2`. In accordance with the E5 convention, all indexed content is prefixed with `passage:` prior to encoding. Embeddings are computed in batches and stored as dense vectors in a Qdrant similarity-search database using cosine distance.

Batch Upserts and Scalability

To support efficient ingestion of large corpora, vectors are upserted to Qdrant in batches (500 points per request for both text chunks and KPI facts). This batching strategy controls memory use and improves ingestion throughput without changing retrieval semantics.

A.2 Claim-Conditioned Evidence Retrieval

This stage operationalizes evidence retrieval for each generated claim by querying the Qdrant vector database. Given a claim and its associated report identifier (`report_s3_key`), the retriever returns the top-k most similar candidate evidence units drawn from two collections: the narrative **Text Collection** (chunked passages) and the structured **KPI Collection** (pseudo-sentences from tables, numeric lines, and iXBRL facts). Retrieval is constrained to the originating report via payload filtering to prevent cross-document leakage and to ensure that verification is strictly grounded in the source document.

Embedding Model and Query Encoding Convention. All retrieval queries are embedded using the same bi-encoder used for indexing, `intfloat/e5-base-v2`, ensuring that claims and evidence candidates lie in the same vector space. In accordance with the E5 query convention, each claim is encoded with a `query:` prefix prior to embedding. Concretely, the retrieval query string is constructed as:

```
query: {claim}
```

The resulting dense vector is used as the Qdrant query embedding.

Within-Report Filtering for Evidence Grounding. To guarantee that retrieved evidence originates from the same report as the claim, a strict payload filter on `s3_key` is applied during Qdrant search. This constraint ensures that retrieval does not exploit semantically similar passages from other companies or years, preserving an auditable verification setting.

Dual-Collection Retrieval and Candidate Merging. For each claim, two Qdrant collections are independently queried: (1) `reports_text` (narrative chunks) and (2) `reports_kpi` (structured KPI facts). Each collection is searched with the same query embedding and the same within-report filter, returning the top-k scored points per collection. The candidates from both collections are then concatenated and globally re-ranked by similarity score, after which the final top-k candidates are retained. This design ensures that retrieval can surface either narrative evidence (e.g., methodological explanations) or structured numeric disclosures (e.g., emissions totals) depending on which yields higher semantic alignment with the claim.

Candidate Record Schema and Minimal Evidence Payload. Each retrieved Qdrant point is transformed into a lightweight candidate record containing (i) a stable point identifier, (ii) collection provenance, (iii) similarity score, and (iv) minimal evidence fields required for downstream verification and inspection. Specifically:

- **Text candidates** include `chunk_id` (the chunk ordinal within the report).
- **KPI candidates** include `sentence` (the canonical pseudo-sentence representing a table cell, iXBRL fact, or numeric line).

The `s3_key` is propagated to each candidate record as the report provenance field. This structure preserves traceability while keeping retrieval outputs compact for storage and subsequent training.

A.3 Cross-Encoder Training with Multiple Instance Learning (MIL)

Training Inputs and Split Files. Training consumes split-specific JSON artifacts stored in S3 under `results/splits/`, e.g.,

```
results/splits/claims_train.json
results/splits/claims_dev.json
```

Each item contains a claim string, its originating report identifier (`report_s3_key`), an optional gold label, and a list of retrieval candidates (`candidates`) containing either (i) text chunk references (`chunk_id`) or (ii) KPI pseudo-sentences (`sentence`) for structured evidence. Labels are normalized into a two-class space using a fixed mapping: `support` $\rightarrow$ `support`, and `notsupport`/variants $\rightarrow$ `not support`. Items with missing or non-mappable labels are excluded to avoid ambiguous supervision.

Evidence Bag Construction (Retrieved + In-Document Negatives). For each claim, we construct an MIL *bag* of evidence candidates. The bag includes: (i) the top-k retrieved candidates from Stage 1 (with $k = 5$), and (ii) an additional set of hard negatives sampled from the *same report* ($n = 2$) to increase discriminative pressure. Negative chunks are sampled from the report's chunk list excluding chunk IDs already present among retrieved text candidates. This yields bags that preserve topical similarity while discouraging shallow lexical matching.

Evidence entries are materialized as raw text strings at training time:

- For KPI candidates, the canonical pseudo-sentence (`sentence`) is used directly.
- For text candidates, `chunk_id` is resolved by re-loading the report from S3 and applying the same whitespace tokenization and sliding-window chunking used in Stage 0 (400-token chunks, 100-token overlap).

To improve throughput, the implementation caches (i) decoded split JSONs, (ii) report bytes downloaded from S3, and (iii) tokenized chunk lists per report, with a bounded in-memory cache to control RAM usage.

Model Architecture and Partial Fine-Tuning. The verification model is initialized from `microsoft/deberta-v3-base` as a sequence-pair classifier with two output labels (`support`, `not support`). To reduce compute and improve training stability on limited data, we employ partial fine-tuning: all parameters are frozen by default, then only the classifier head and the top N transformer layers are unfrozen ($N = 4$). This strategy concentrates learning capacity near the output layers, where task-specific decision boundaries are formed, while preserving the general linguistic and entailment representations learned during pretraining.

Pair Encoding and MIL Collation. Training operates over bags but the cross-encoder consumes individual pairs. For each bag, we create one input pair per evidence string:

```
[CLS] claim [SEP] evidence [SEP]
```

Pairs are tokenized with the DeBERTa tokenizer using truncation and fixed-length padding to `MAX_SEQ_LEN=512`. The dataloader collate function flattens all pairs across bags into a single tensor batch while emitting two auxiliary tensors: (i) `bag_ids`, mapping each pair back to its bag index, and (ii) `bag_labels`, storing the label per bag. This enables bag-level pooling and loss computation while still using standard transformer forward passes on pair batches.

MIL Pooling Objective and Optimization. Optimization uses AdamW with weight decay (0.01), excluding bias and LayerNorm weights from decay. A linear learning-rate schedule with warmup is applied (warmup = 10% of total steps). To accommodate long sequences and variable bag sizes under limited GPU memory, training uses: `BATCH_SIZE=4` bags per step with gradient accumulation (`GRAD_ACCUM_STEPS=8`) to achieve a larger effective batch size. Mixed precision is enabled via CUDA autocast and a gradient scaler.

Bag-Level Evaluation. Validation is performed at the bag level. For each bag, instance logits are pooled using the same log-sum-exp operator to obtain bag logits, after which the predicted label is the argmax over the pooled logits. Bag-level accuracy is computed as the fraction of bags whose predicted label matches the bag label. This evaluation aligns with the MIL training objective and avoids overstating performance based on individual evidence pairs that are not directly supervised.

Checkpointing and Model Publishing. After training completes, the fine-tuned model and tokenizer are saved locally and uploaded to S3 under:

```
results/models/deberta_finetuned_mil/
```

The upload routine mirrors the local directory structure, enabling reproducible loading for later verification runs.

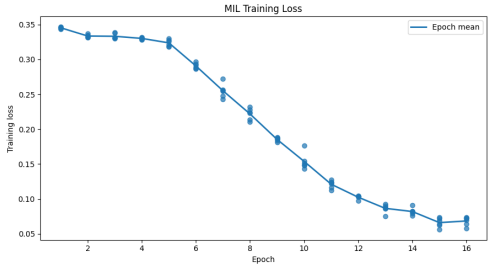

Figure 2: Training loss over optimization steps for MIL fine-tuning of the DeBERTa cross-encoder.

References

Andreas Hanselowski, Hao Zhang, Zile Li, Daniil Sorokin, Benjamin Schiller, Claudia Schulz, and Iryna Gurevych. 2018. Ukp-athene: Multi-sentence textual entailment for claim verification.

J. Li and 1 others. 2025. Optimizing large language models for esg activity detection in financial texts. *arXiv preprint arXiv:2502.21112*.

Max Welling Maximilian Ilse, Jakub M. Tomczak. 2018. Attention-based deep multiple instance learning. *Proceedings of NAACL-HLT 2019*.

Aleksandra Piktus Fabio Petroni Vladimir Karpukhin Naman Goyal Heinrich Küttler Mike Lewis Wen-tau Yih Tim Rocktäschel Sebastian Riedel Douwe Kiela Patrick Lewis, Ethan Perez. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks.

Jianfeng Gao Weizhu Chen Pengcheng He, Xiaodong Liu. 2021. Deberta: Decoding-enhanced bert with disentangled attention. *Journal of Machine Learning Research*.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks.

T. Schimanski and 1 others. 2024. Bridging the gap in esg measurement: Using nlp to quantify environmental, social, and governance communication. *Journal of Behavioral and Experimental Finance*.

Y. Seki and 1 others. 2024. Ml-promise: A multilingual dataset for corporate promise verification. In *Proceedings of the Conference on Computational Linguistics*.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification.

U.S. Securities and Exchange Commission. 2025. SEC EDGAR Company Filings Search. `https://www.sec.gov/search-filings`. Accessed: 2025-12-10.

Liang Wang, Nan Yang, Xiaolong Huang, Furu Huang, Rangan Majumder, Ming Wang, Bing Zhou, and Jimmy Lin. 2022a. Text embeddings by weakly-supervised contrastive pre-training.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022b. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*.

W. Zhang and 1 others. 2025. Corporate greenwashing detection in text: A survey. *arXiv preprint arXiv:2502.07541*.

X. Zou and 1 others. 2025. Esgreveal: An llm-based approach for extracting structured data from esg reports. *Journal of Cleaner Production*.